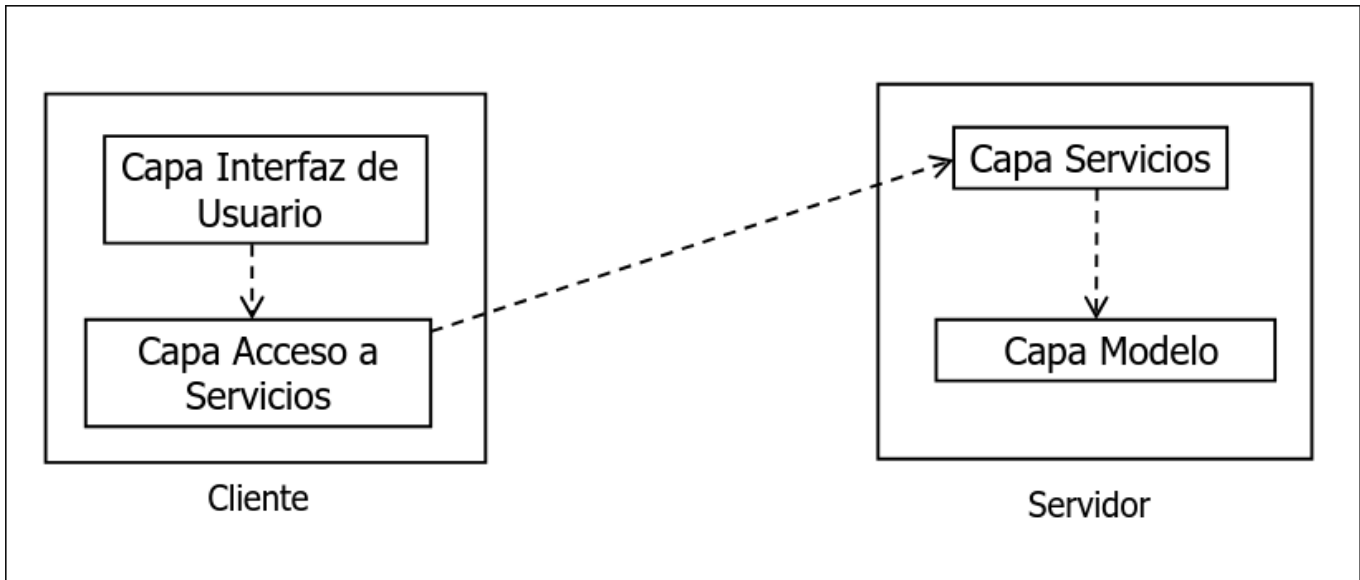


Diseño por capas

En el caso de estudio usaremos el diseño por capas para ocultar las tecnologías de acceso e implementación de los servicios



Capa **Interfaz de Usuario** (o lógica de la aplicación cliente)

- Implementa la interfaz de usuario
- No depende de la tecnología de acceso al servicio

Capa **Acceso a Servicios**

- Implementa el acceso a los servicios
- Ofrece una API que oculta la tecnología usada para acceder a los servicios.

Capa **Servicios**

- Utiliza una tecnología para implementar los servicios.
- Delega en la capa Modelo.

Capa **Modelo**

- Implementa la lógica de la aplicación.
- No depende de la tecnología de implementación de los servicios.

El uso de diseño por capas en este caso tiene las ventajas habituales:

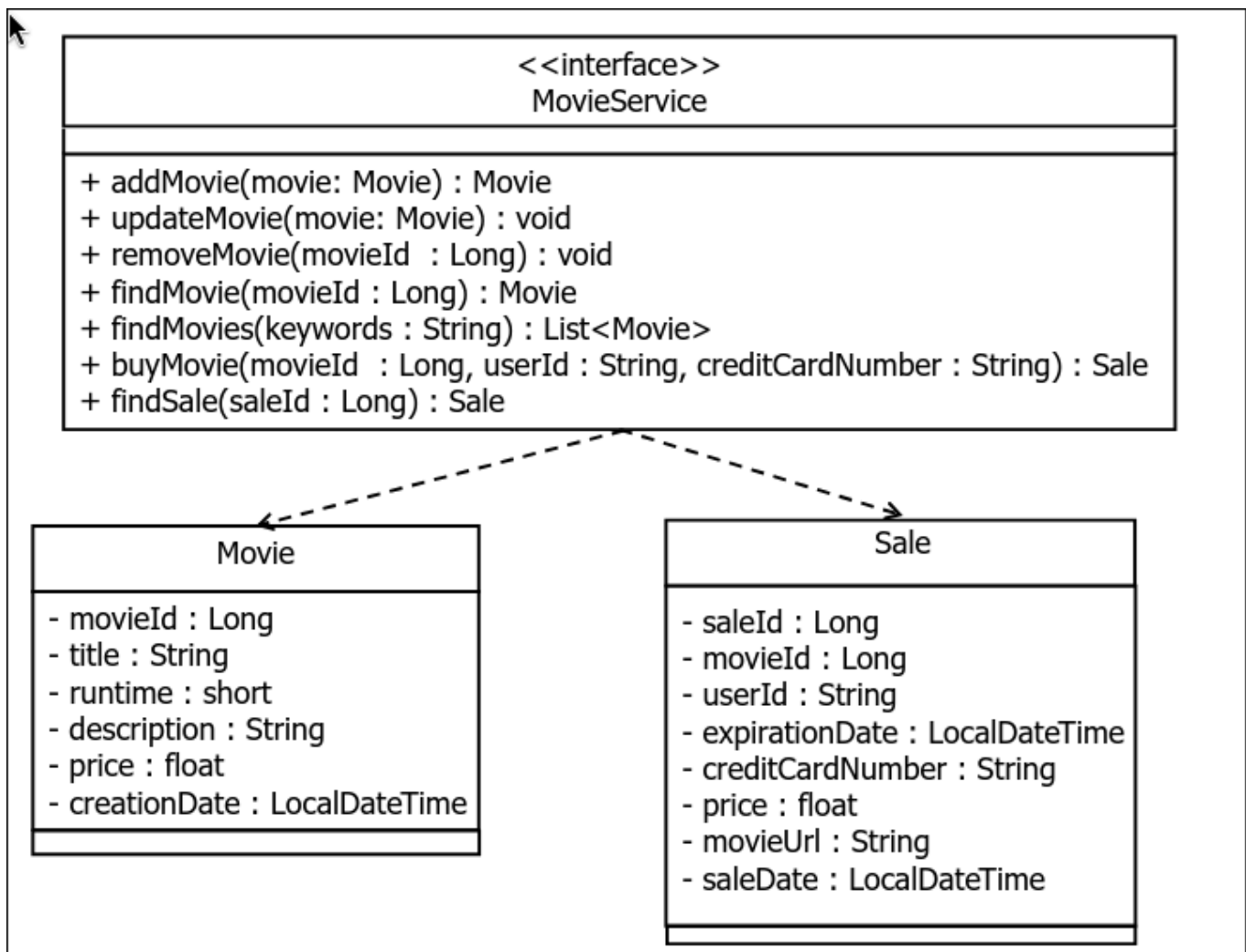
- La persona que se encarga del desarrollo de las capas IU y Modelo no tienen porqué saber nada sobre las tecnologías de los servicios.
- Cada capa puede ser desarrollada en paralelo

- Se facilita el mantenimiento del software.
 - Cambios en la implementación de una capa no conllevan cambios en el resto de capas.

Así como los riesgos habituales:

- El software es más complejo
 - Si el software a construir es muy sencillo puede no valer la pena.
- A veces, el que el modelo conozca los detalles de la tecnología de implementación del servicio podría permitir ciertas optimizaciones.

Interfaz de la Capa Modelo



Capa Servicios

Expone remotamente la capa modelo que creamos en "[3. Diseño e implementación de la capa modelo](#)", sin embargo, los servicios no expondrán directamente los objetos **Movie** y **Sale** del modelo.

La razón es que en este ejemplo se asume que el acceso remoto lo van a realizar aplicaciones externas.

- La fecha de creación de la película en la BD es un dato interno que no se desea exponer a aplicaciones externas.
- En cuanto a las ventas, los clientes externos sólo necesitan los identificadores de venta y película, la fecha de expiración y la url para ver la película.

Por tanto, los servicios utilizarán dos objetos alternativos: `ServiceMovieDto` y `ServiceSaleDto`.

DTO: Data Transfer Object. Objeto para transferir datos entre aplicaciones. Encapsula diferencias con los objetos internos de cada aplicación.

- Porque no son relevantes externamente (nuestro caso)
- Porque queremos objetos más "gruesos" para minimizar llamadas remotas.
- Ejemplo:
 - Supongamos que tenemos también información de actores, los actores tiene su propia tabla en la BD, su clase y su DAO, entre otros casos de uso para manejar información de actores, el modelo ofrece el método `getActorsByMovie`, que recibe un `movieId` y devuelve una lista de actores. Si sabemos que la mayoría de los clientes, al buscar información de películas van a querer los actores, podríamos hacer que el objeto `ServiceMovieDto` incluyese ya la lista. La operación `findMovies` del servicio invocaría a `findMovies` del Modelo y, para cada invocaría a `getActorsByMovie` para obtener los actores.

Esto ilustra el hecho común de que a las aplicaciones externas se les expone una API diferente que a las internas.

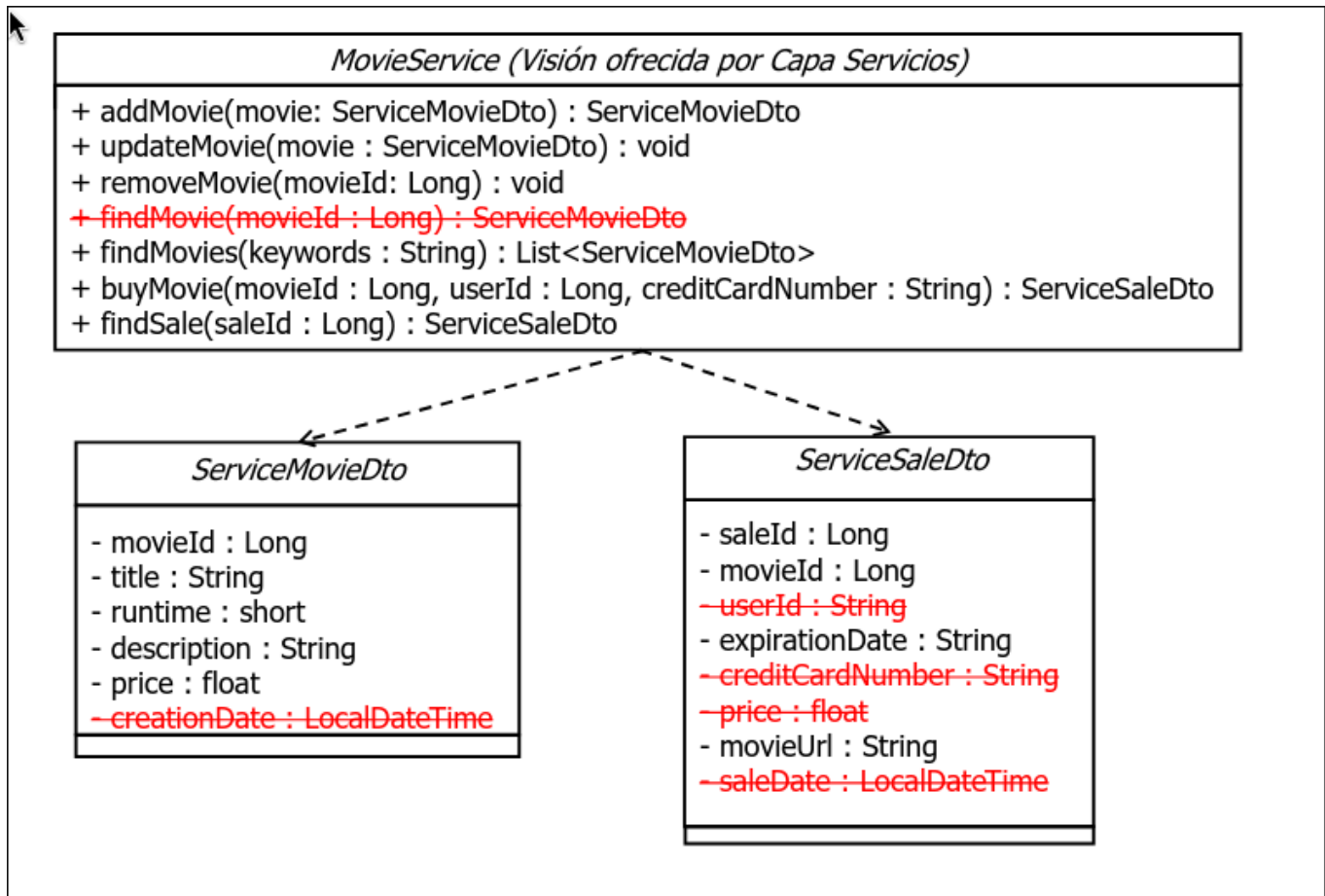
- Diferencias en los objetos expuestos.
- No exponemos la operación `findMovie`
- En casos más complejos podrían ser necesarias validaciones o pasos adicionales antes de invocar a la capa modelo.

En nuestro caso, asumimos que las aplicaciones internas que necesiten acceder a más datos de las películas y ventas que los expuestos por la capa Servicios diseñada para las aplicaciones externas, accederían a través de otra capa Servicios con una API menos restringida.

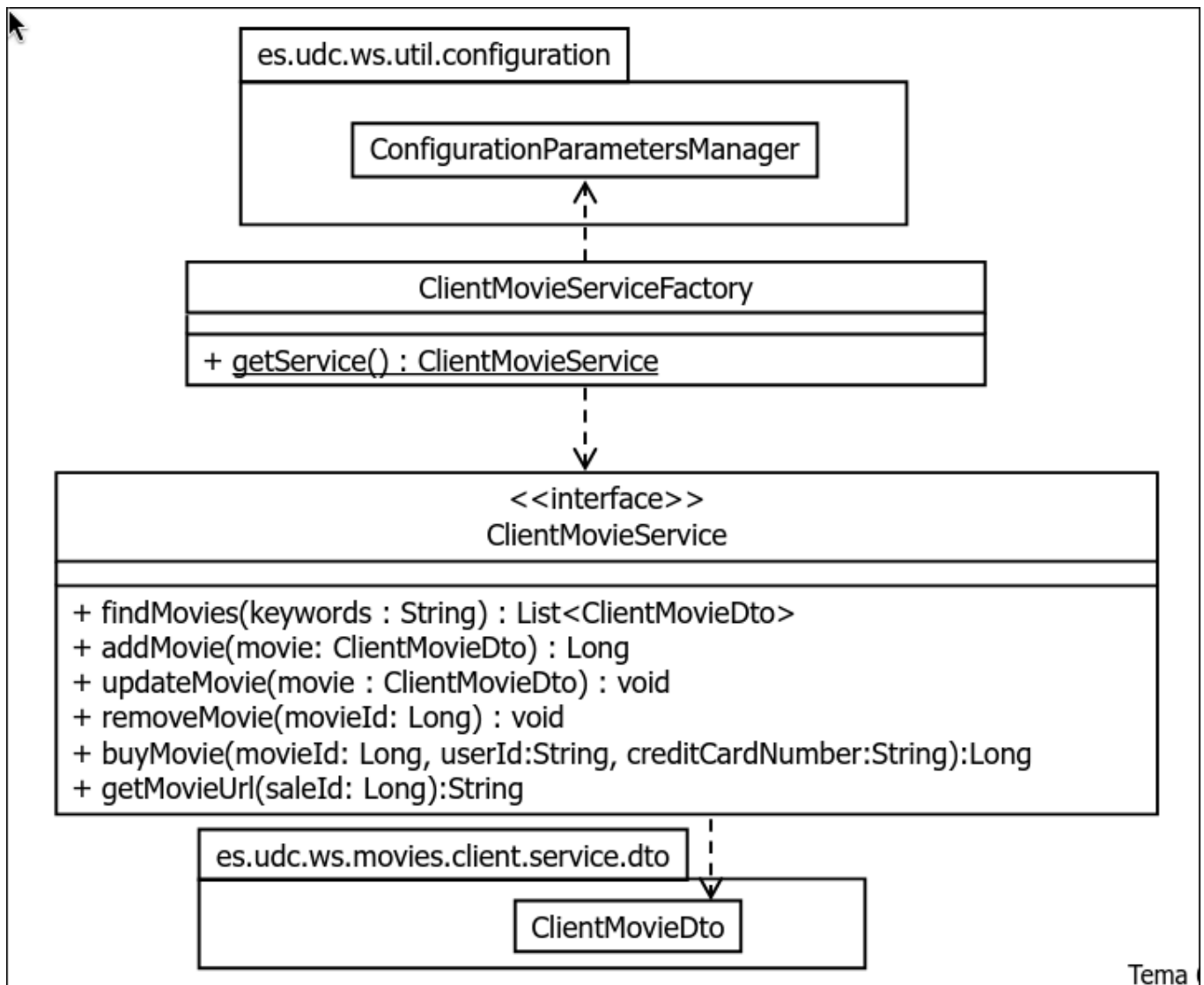
NOTA: En `ServiceSaleDto` la propiedad `expirationDate` es tipo `String` en lugar de `LocalDateTime`

- La fecha se envía al cliente como un string.

Interfaz de la Capa Servicios



Capa Acceso a Servicios



Tema 1

Define una **sencilla API** para acceder al servicio, **oculta la tecnología** usada para acceder al servicio.

Paquete `es.udc.ws.movies.client.service`

`ClientMovieService`

- Patrón Fachada
- Interfaz con un método por cada una de las operaciones del servicio.

`ClientMovieServiceFactory`

- Patrón Factoría
- Permite construir una instancia de `ClientMovieService` sin que el llamador necesite conocer la clase que implementa el interfaz.

• `getService`

- Lee de un fichero de configuración el nombre de la clase que implementa la interfaz
- Construye una instancia de esa clase.

`ClientMovieService` + `ClientMovieServiceFactory`:

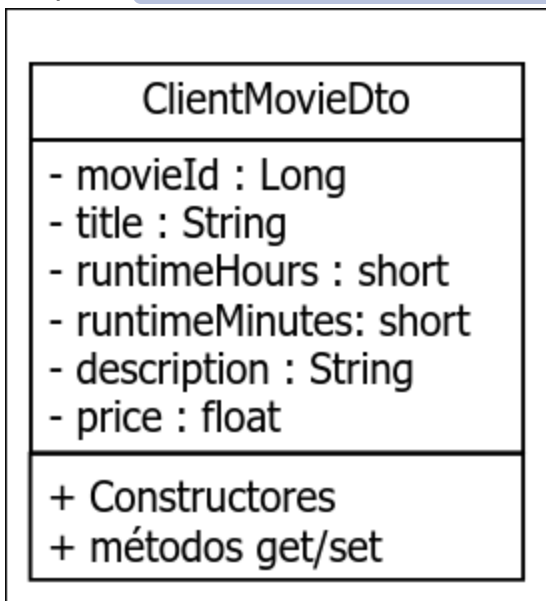
- Además de las anteriores ventajas, sencillez y ocultación de la tecnología, permiten que el cliente pueda acceder a distintas implementaciones del servicio sin necesidad de recompilarlo.
 - Basta especificar en la configuración la implementación de `ClientMovieService` que se desea.
- El caso de estudio proporciona dos implementaciones de `ClientMovieService`: una para acceder a un servicio web REST y otra para acceder a un servicio Thrift.
 - En ambos casos, los clientes son los mismos (**no se han implementado dos veces**)

Ejemplo de uso

```
ClientMovieService service = ClientMovieServiceFactory.getService();  
List<ClientMovieDto> movies = service.findMovies("batman");
```

La aplicación cliente usará el `runtime` dividido en horas y minutos, por lo que usa su propio objeto para representar la información de películas.

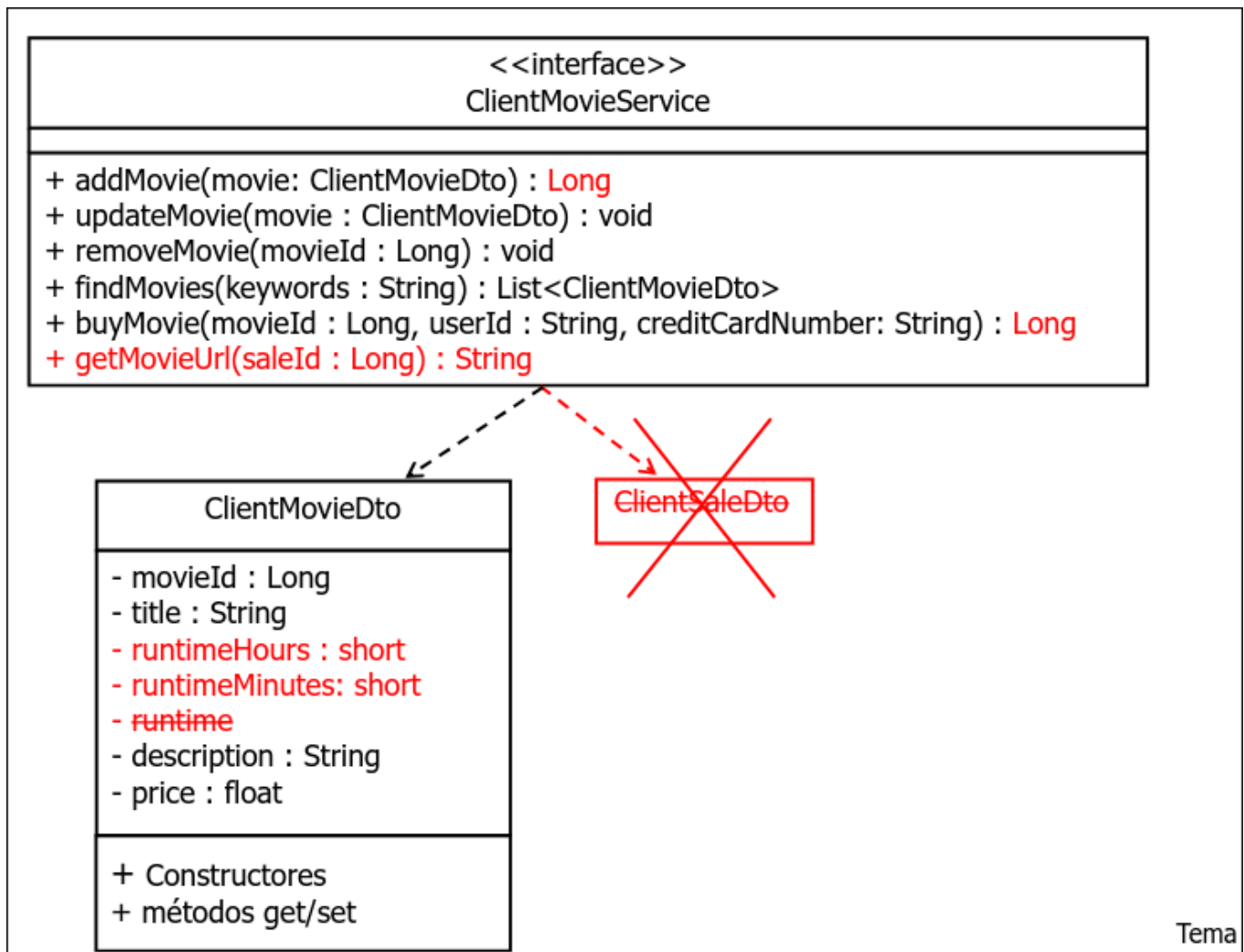
Paquete `es.udc.ws.movies.client.service.dto`



En los casos reales, lo habitual es que cada cliente use sus propios objetos, ya que:

- Permite adaptar los objetos a lo que el cliente necesita.
- Es más natural tener objetos con sólo lo que necesitas.
- Ejemplo: Al escribir esta diapositiva, la API de Facebook expone 38 campos y 42 relaciones para un usuario, una aplicación que te avise del cumpleaños de tus amigos sólo necesita dos atributos de su objeto User (name y birthdate).

Interfaz de la Capa Acceso a Servicios



Tema

Capa Interfaz de Usuario

Cliente MovieServiceClient

- Añadir película: `MovieServiceClient -a <title> <hours> <minutes> <Description> <price>`
- Borrar película: `MovieServiceClient -r <movieId>`
- Actualizar película: `MovieServiceClient -u <movieId> <title> <hours> <minutes> <description> <price>`
- Buscar películas por keywords en el título: `MovieServiceClient -b <movieId> <userId> <creditCardNumber>`
- Ver película (recuperar venta): `MovieServiceClient -g <saleId>`

NOTA:

- Tenemos un único cliente que puede ejecutarse desde la intranet del servicio o desde Internet. Sin embargo en un caso real, posiblemente habría al menos dos tipos de clientes. Uno para las labores de administración que se ejecutaría normalmente desde la intranet del

servicio y otro para buscar, comprar y ver películas que se usaría normalmente desde Internet.

- El cliente especifica la duración de la película separándola en horas y minutos.

Introducción a los servicios Web REST

Introducción

REpresentational State Transfer. Estilo arquitectónico propuesto por Roy Fielding en 2000. La Web es, sin duda, la aplicación más exitosa de la historia.

REST: Estilo arquitectónico para construir aplicaciones distribuidas inspirado en las características de la Web.

Aunque es independiente de la tecnología, se suele implementar usando directamente HTTP y algún lenguaje de intercambio, como XML o JSON, sin tecnologías adicionales.

- A menudo los servicios REST reales no siguen estrictamente todos los principios del estilo arquitectónico REST.
- A los servicios que sí siguen fielmente todos los principios REST, se les denomina a veces servicios web RESTful.

Recordatorio HTTP

- HTTP: HyperText Transfer Protocol.
- Protocolo cliente/servidor utilizado en la Web.
- Inicialmente construido para transferir páginas HTML, pero puede transferir cualquier contenido.
 - Las peticiones y respuestas HTTP usan las cabeceras Accept y Content-Type para indicar el tipo de MIME que indica la naturaleza de la información que se espera recibir o se envía en el cuerpo, respectivamente.
- Esquema petición/respuesta

Peticiones HTTP

Concepto clave: URL como identificador **global** de recursos.

Una petición HTTP consta de

- Una URL, que identifica al recurso sobre el que actúa la petición.
- Un método de acceso (GET, POST, PUT, ...), que especifica la acción a realizar sobre el recurso.
- Cabeceras
 - Metainformación de la petición que indican información adicional que puede ser necesaria para procesar la petición.

- Cuerpo del mensaje (opcional)
 - Solo con algunos métodos de acceso.

Métodos de acceso:

- GET
 - Solicita una representación del recurso especificado (e.g. página HTML)
 - Es seguro
 - Semántica esencialmente de sólo lectura
 - El cliente no pide ni espera que se produzcan cambios en el servidor como resultado de realizar una petición sobre un recurso utilizando este método.
 - Un cliente puede utilizarlo sin miedo a causar efectos indeseados.
 - Puede especificarse parámetros en la URL (consultas). Se especifican como pares campo=valor, separados por el carácter '&' **ESTOS VALORS VAN CODIFICADOS.
- PUT
 - Carga en el servidor una representación de un recurso (que podría existir previamente o no)
 - La nueva representación del recurso va en el cuerpo de la petición.
 - No es seguro.
- Delete
 - Elimina el recurso especificado
 - No es seguro
- POST
 - Envía datos al recurso indicado para que los procese
 - Los datos van en el cuerpo de la petición.
 - Puede utilizarse para solicitar la ejecución de cualquier acción en el servidor.
 - No es seguro

Peticiones GET, PUT, DELETE deben ser idempotentes

- Múltiples peticiones iguales deben tener el mismo efecto en el servidor que una sola.

Las peticiones POST no tiene porque ser idempotentes

Cabeceras de la petición

- Tipo MIME de datos enviados en el cuerpo
- Tipo MIME de datos esperados (e.g HTML, XML, JSON, ...)
 - El cliente puede especificar qué formatos entiende y en qué formato prefiere la representación.

- Encoding esperado.
- Lenguaje esperado.
- Peticiones condicionales (If-Modified-Since, Etags)
 - Solicita una representación sólo si ha cambiado desde un momento determinado.
 - Cachés en el cliente.
- Credenciales de autenticación / autorización.
- Información para proxies.
- Agente del usuario.

Una respuesta HTTP contiene:

- Código de Status [Lista Completa](#)

Código	Significado
200	OK
201	Created
400	Bad Request
403	Forbidden
404	Not Found
500	Internal Error

- Cabeceras. Metainformación de la respuesta.
- Cuerpo del mensaje (opcional)
 - GET: representación del recurso invocado.
 - POST: vacío o con el resultado del procesamiento realizado por el servidor.
 - PUT: normalmente, el cuerpo viene vacío.
 - DELETE: normalmente, el cuerpo viene vacío.
 - En caso de error, puede incluir información con más detalle.

Las cabeceras especifican metainformación adicional sobre las repuestas, entre otras:

- Tipo MIME de datos devueltos.
- Encoding de la respuesta.
- Lenguaje de la respuesta
- Antigüedad de la respuesta
- Longitud de la respuesta
- Control de caché: si el recurso se puede cachear o no, tiempo de expiración (si se puede cachear), momento de última modificación, ...

- Control de reintentos.
- Identificación del servidor

Características de los Servicios REST

Los servicios REST se estructuran de forma similar a un sitio de la WWW, sin embargo, en lugar de páginas HTML, normalmente accederemos a información estructurada (e.g. películas, clientes, etc.). Al igual que la WWW está compuesta de muchos sitios web autónomos, los servicios REST se consideran autónomos entre sí. Al igual que en la WWW, un servicio REST puede referencias a otro (links) y habrá una serie de servicios generales (e.g cachés) que pueden funcionar sobre cualquier servicio.

- Cliente -> Servidor
- Servicios sin estado ("stateless").
- Recursos y representaciones
- Interfaz uniforme.
- Sistema en capas: Intermediarios.
- Hipermedia [x]
- Representaciones autodescriptivas. [x]

Servicios "Stateless"

Cliente-Servidor

- Clientes invocan directamente URLs para acceder a recursos.

El servidor no guarda información de estado para cada cliente.

Ejemplo de servicio con estado: FTP

1. El servidor guarda el directorio en el que está cada cliente
2. El resultado de la ejecución de un comando (e.g get, para obtener un fichero) depende de ese estado.
3. Si el fichero `file.pdf` está en la ruta `/docs` y el cliente está en ese directorio el comando `get file.pdf` transferirá el fichero. En otro caso, dará error.

Ejemplo de servicio sin estado: HTTP

- Cada petición de un cliente debe contener toda la información necesaria para que el servidor la responda.
- Ejemplo `https://www.fileserver.com/docs/file.pdf`

Ventajas de los servicios sin estado

- Replicación del servicio en múltiples máquinas es muy sencilla: basta usar un balanceador de carga que dirige cada petición a una instancia del servicio.
 - En un servicio con estado, es necesario o bien garantizar que todas las peticiones de la misma "sesión" van al mismo servidor o bien usar un servidor de sesiones.
- El servidor no necesita reservar recursos para cada sesión.
 - Disminuye los recursos que necesita el servicio.
 - Si el cliente falla o se cae en el medio de la sesión, el servidor no tiene que preocuparse de detectarlo ni de liberar recursos.

En general: mejora la escalabilidad de los servicios.

Inconveniente: puede ser necesario enviar información adicional con cada petición

Recursos y Representaciones

Una aplicación REST se compone de recursos. Suelen corresponderse con las entidades persistentes.

Hay dos tipos de recursos:

- Colección: identifican una serie de recursos del mismo tipo (películas en nuestro ejemplo. clientes de una empresa, ...).
- Individuales: identifican un elemento concreto (una película, un cliente, ...)

Cada recursos es identificado mediante un identificador único y global.

- <http://www.movieprovider.com/movies/>
 - Recurso colección que representa todas las películas.
- <http://www.movieprovider.com/movies/3>
 - Una película de movieprovider.com
- <http://www.acme.com/costumers/01235>
 - Un cliente de acme.com

Los identificadores (URLs) son globales

- Todo recurso tiene un nombre único a nivel inter-aplicación
- Por tanto, cualquier servicio puede referenciar y acceder a un recurso de cualquier otro servicio.

Al invocar la URL (usando GET), el cliente obtiene una **representación** del recurso.

La representación debe contener información útil sobre el recurso

- Recursos colección: normalmente lista de los elementos de la colección con información resumen y un enlace para obtener información completa.

- Recursos individuales: datos del elemento individual (e.g. datos de la película, datos del cliente, ...)

La representación de un recurso puede variar en el tiempo.

- El identificador está ligado al recurso, no a la representación.
- Si cambian los datos de un cliente de Acme, cambiará la representación.
 - La URL apuntará siempre a la representación actual del recurso.

Ejemplo representación recurso colección:

```
<?xml version="1.0" encoding="UTF-8"?>
<customers xmlns="http://www.acme.com/restws/customers">

  <customer>
    <cid>1234</cid>
    <cname>Roadrunner</cname>
    <link href="http://www.acme.com/restws/customers/1234"/>
  </customer>
  <customer>
    <cid>1235</cid>
    <cname>Coyote</cname>
    <link href="http://www.acme.com/restws/customers/1235"/>
  </customer>
  ...
</customers>
```

Ejemplo representación recurso individual

```
<?xml version="1.0" encoding="UTF-8"?>
<customer xmlns="http://www.acme.com/restws/customers">
  <cid>1234</cid>
  <cname>Roadrunner</cname>
  <address> Monument Valley, 5 </address>
  <description> He is fast</description>
  <rating> Good </rating>
</customer>
```

Interfaz Uniforme

Las operaciones disponibles sobre los recursos son siempre las mismas. Las normas REST no dicen cuáles deben ser esas operaciones, sólo que deben ser siempre las mismas y funcionar igual en todos los servicios. Los tipos de respuesta (tanto de éxito como de error) que pueden devolver estas operaciones deben estar también estandarizados. Nuevamente, REST no

impone un conjunto de códigos de respuesta concretos, sólo que deben ser los mismos para todos los servicios.

En la práctica, se utiliza HTTP

- GET
 - Acceso a representaciones.
 - Consultas sobre recursos colección. `http://www.movieprovider.com/movies?keywords=Dark+Knight`
 - Es segura y es idempotente.
- PUT
 - Reemplaza la representación de un recurso.
 - Si la URL no existe, y el servicio lo permite, crea el recurso.
 - No suele permitirse sobre recursos colección.
 - No es segura y es idempotente.
- DELETE
 - Borra el recurso
 - No suele permitirse sobre recursos colección
 - No es segura y es idempotente.
- POST
 - Sobre recursos colección, suele usarse para crear un nuevo recurso en la colección.
 - El servidor devuelve la URL del nuevo elemento usando una cabecera estándar HTTP.
 - Sobre recursos individuales (o colección), puede usarse para modelar operaciones no seguras que no encajen con los otros métodos -> **overloaded POST**
 - En este caso la petición debe incluir en algún otro lugar (en la URI, cabeceras o cuerpo) información sobre la operación a invocar sobre el recurso.
 - Semejante a estilo RPC (no sigue principios de interfaz uniforme).
 - No es segura y puede no ser idempotente.

Intermediarios

El uso de una interfaz uniforme permite que haya intermediarios entre el cliente y el servicio que proporcionen funcionalidad adicional sin necesidad de saber nada adicional sobre ellos. Los intermediarios son transparentes para el cliente y el servicio. Esto es posible porque todos los servicios soportan una interfaz uniforme (operaciones, códigos de respuesta, cabeceras).

Diseño e implementación de un servicio REST

Protocolo REST

Seguiremos la misma aproximación que la mayoría de servicios web REST reales.

- URLs únicas y globales para cada recurso.
- Uso consistente de GET, PUT, POST y DELETE
- Uso consistente de los códigos de respuesta HTTP
- Uso de algunas cabeceras estándar HTTP
- En la asignatura Integración de Aplicaciones se profundizará sobre el uso de estos conceptos.

Recursos

- `/movies`
 - GET
 - Lista todas las películas
 - La información de cada película se representa en JSON en formato del apartado 5.4
 - El parámetro keywords permite filtrarlas por palabras clave en el título: `/movies?keywords=Dark+Knight`
 - Devuelve el código `200 OK`
 - POST
 - Añade una película.
 - La película se envía en el cuerpo de la petición en el formato JSON del apartado 5.4
 - Devuelve el código de respuesta HTTP: `201 Created`
 - Devuelve la URL de la nueva película usando la cabecera estándar HTTP `Location`.
 - El cuerpo de la respuesta devuelve la nueva película creada.
 - PUT
 - Modifica la película.
 - La película se envía en el cuerpo de la petición en el formato JSON del apartado 5.4
 - El cuerpo de la respuesta va vacío.
 - Devuelve el código `204 No content`
 - DELETE
 - Borra la película.
 - El cuerpo de la respuesta va vacío.
 - Devuelve el código `204 No content`
- `/sales`
 - Post

- Añade una nueva venta
- Los datos de la venta (`userId`, `movieId`, `creditCardNumber`) se reciben como parámetros.
- Devuelve el código de respuesta HTTP: `201 Created`
- Devuelve la URL de la nueva venta usando la cabecera estándar HTTP `Location`.
- El cuerpo de la respuesta devuelve los datos de la nueva venta creada.
- `/sales/{id}`
 - GET
 - Obtiene la información de la venta
 - La información de cada venta se representa en JSON
 - No es posible borrar ni modificar ventas una vez producidas.

Errores: se utilizan los códigos HTTP más próximos a la semántica de la respuesta.

- Parámetros incorrectos: `400 Bad Request`
 - Similar a `InputValidationException`
- Recurso no existe: `404 Not Found`
 - Similar a `SaleExpirationException`
- Recurso existió pero ya no existe: `404 Gone`
 - Similar a `SaleExpirationException`
- Se deniega el acceso a la acción solicitada: `403 Forbidden`
 - Similar a `MovieNotRemovableException`
- Error interno de ejecución: `500 Internal Error`

Pero el cuerpo del mensaje puede llevar información adicional.

- Normalmente un mismo código de error estará asociado a varias excepciones -> Debe indicarse de alguna forma a cuál se refiere en cada caso.
- Representación de los datos de la excepción.

Consideraciones Diseño REST

Creación de recursos con POST

- La URL en la cabecera `Location` permite al cliente conocer la URL del nuevo recurso creado para referirse a él más tarde.
- Usar una cabecera estándar permite proporcionar semántica para cualquier intermediario y cliente, aunque no conozcan los formatos de nuestro servicio.

- Devolver completo el nuevo recurso creado en el cuerpo es útil si creemos que el cliente va a utilizarlo de inmediato, pero si la representación puede ser grande y no es seguro que el cliente la necesite inmediatamente, puede ser mejor enviar sólo el identificador.
- ¿Parámetros o representación ad-hoc en el cuerpo?
 - Usar parámetros es más simple pero está limitado al envío de pares campo/valor.

Códigos de respuesta y de error

La ventaja de usar códigos estandarizados es que cualquier cliente o intermediario que utilice ese estándar conoce la semántica de la respuesta.

Por ejemplo, se puede reintentar una petición correspondiente a una operación idempotente que ha devuelto un código de error sin miedo a causar efectos indeseados. Además si se sabe si el error es temporal o permanente se sabe si el reintento puede funcionar o no.

En caso de error, el cuerpo del mensaje puede llevar información adicional para los clientes específicos del servicio.

Aplicaciones Web Java EE / Jakarta EE

En Java EE / Jakarta EE, las **aplicaciones Web** se instalan en **servidores (contenedores) de aplicaciones**

La API de programación Web que ofrece el servidor de aplicaciones está estandarizada.

Una aplicación web Java EE puede instalarse en cualquier servidor de aplicaciones Java EE.

Las aplicaciones Web se distribuyen en ficheros WAR

- Un fichero WAR (Web ARchive) es un fichero JAR con una estructura de directorios estandarizada.
- Contiene clases objeto, librerías, ficheros de configuración y otras abstracciones propias de una aplicación Web.

Las APIs de programación Web Java no están pensadas para devolver sólo HTML/XHTML, sino cualquier tipo de información sobre HTTP. En consecuencia, se pueden utilizar para implementar servicios Web. Utilizaremos la **API de Servlets** para implementar Servicios Web REST.

Visión global del Framework de Servlets

Para la implementación de la capa servicios usaremos la API de Servlets.

Un Servlet es una clase que:

- Se asocia a una o varias URLs
- Cuando el servidor recibe una petición sobre esa URL, invoca al servlet asociado.
- El servlet
 - Accede al contenido de la petición

- Ejecuta el código deseado para obtener la respuesta.
- Codifica la respuesta en el formato deseado.
- La respuesta devuelta por el servlet será la respuesta enviada por el servidor de aplicaciones al cliente.

El desarrollador implementa un servlet extendiendo de `HttpServlet` y redefine los métodos `doXXX` necesarios, según el tipo de peticiones que vaya a aceptar su servlet.

Cuando el servidor de aplicaciones recibe una petición dirigida a un servlet, invoca la operación correspondiente en función del método de la petición.